03–06 – Porucha zničí obsluhu nebo počká

Zadání

⚠ Původní zadání se nedochovalo, jen studentský přepis (slovensky):

> Petri MM1 V intervaloch exponenciálně se středem 8 hodin sa objaví porucha ktorá buď:
> - zničí práve prebihajúcu obsluhu - alebo počká na príchod následujúcej obsluhy a tú zničí
>
> + SIMLIB kód Modely MUSIA byť izomorfné.

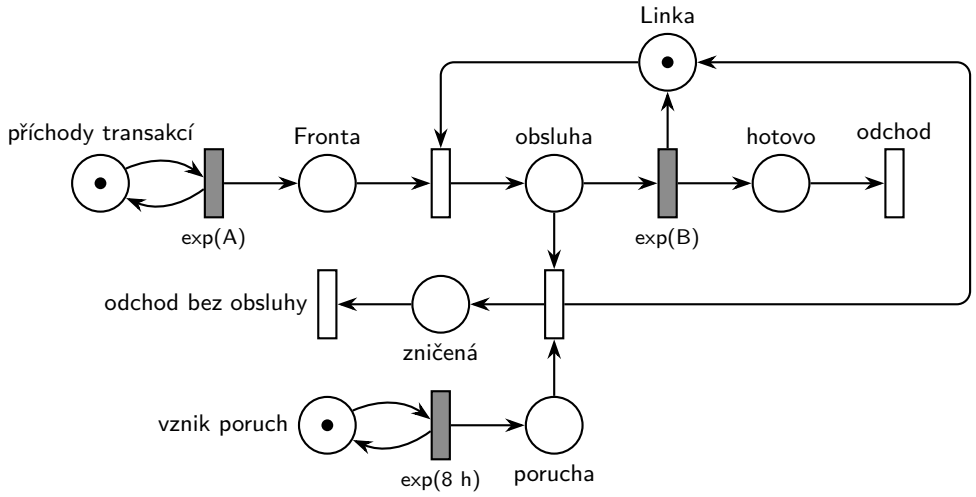
Podobné úlohy:

- 03–01 – Porucha zruší obsluhu: porucha bez obsluhy zanikne
- 03–08 – Porucha prodlouží další obsluhu: porucha čeká na další obsluhu a prodlouží ji
- 09–11 – Kopírovací centrum s poruchami: porucha ničí rozpracované obsluhy

Interpretace

- **M/M/1:** *příchody transakcí* ve smyčce s `exp(A)`, *Fronta*, obsazení *Linky*, obsluha `exp(B)`, která *Linku* vrací, a *odchod*. Časy příchodů a obsluhy zadání neuvádí, proto `exp(A)` a `exp(B)` (v simulaci v minutách).
- **Poruchy** vznikají v intervalech `exp(8 h)`: místo *vznik poruch* ve smyčce s časovaným přechodem, každé provedení vloží značku do místa *porucha*.
- **Stačí jediný přechod:** *porucha* + obsluhovaná transakce (místo *obsluha*) → *Linka* + *zničená*. Když linka obsluhuje, provede se hned. Když neobsluhuje, porucha zůstane v místě *porucha* a přechod se provede ve chvíli, kdy další transakce obsadí linku. Obsluha je tedy zničena hned na začátku.
- **Oproti 03–01 tu chybí přechod *porucha bez účinku*.** Právě ten by poruchu při volné lince zahodil. Bez něj porucha „počká“: žádný jiný přechod z místa *porucha* nevede.
- **Priorita není potřeba:** přechod nemá žádného okamžitého soupeře a před časovaným `exp(B)` má okamžitý přechod přednost vždy (slajd 139).
- **„Zničí obsluhu“:** transakce se neobslouží a odchází ze systému (*zničená* → *odchod bez obsluhy*). Porucha nemá dobu trvání, linka je hned volná.
- **Více čekajících poruch** (druhá přijde dřív, než začne další obsluha) zničí každá jednu další obsluhu. Při exp(8 h) je to vzácné. Kdyby měly splývat v jednu, použije se sloučení jako v 03–08.
- **Izomorfní modely:** každému přechodu sítě odpovídá krok v procesu transakce nebo v události poruchy (viz SIMLIB).

Petriho síť



Časté chyby

- Přechod *porucha bez účinku* převzatý z 03–01: porucha při volné lince zanikne a nepočká.
- Čekající porucha modelovaná vlastním časovaným přechodem nebo testem fronty. Stačí, že porucha zůstane v místě, dokud nemá s čím reagovat.
- Zničení bez vrácení *Linky*.
- Zničená transakce zmizí bez výstupního přechodu.

SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu.

```
// 03–06 Porucha zničí obsluhu nebo počká – model v SIMLIB/C++
#include "simlib.h"

const double A = 10, B = 6;              // symboly ze sítě, v minutách

Facility Linka("Linka");                 // místo Linka (zařízení s kapacitou 1)
Histogram Doba("Doba obsloužené transakce v systému [min]", 0, 10, 20);
int porucha = 0;                         // místo „porucha“: počet čekajících poruch
long zniceno = 0, pocet_poruch = 0;

class Transakce;
Transakce *obsluhovana = nullptr;        // místo „obsluha“ (nullptr = bez značky)

class Transakce : public Process {       // transakce = značka, která prochází sítí
  void Behavior() {
    double prichod = Time;
    Seize(Linka);                        // obsazení: Fronta + Linka → obsluha
    if (porucha == 0) {                  // čekající porucha by obsluhu zničila hned
      obsluhovana = this;
      Wait(Exponential(B));              // časovaný přechod exp(B), porucha ho může zruši
      obsluhovana = nullptr;
    }
    Release(Linka);                      // exp(B) i zničení vrací Linku
    if (porucha > 0) {                   // přechod zničení: porucha + obsluha
      porucha--;
      zniceno++;
      return;                            // „zničená“ → přechod „odchod bez obsluhy“
    }
    Doba(Time - prichod);
```

```
  }                                          // konec Behavior = přechod „odchod“
};

class Porucha : public Event {               // „vznik poruch“ ve smyčce s exp(8 h)
  void Behavior() {
    porucha++;                               // nová značka v místě „porucha“
    pocet_poruch++;
    if (obsluhovana)                         // probíhající obsluha:
      obsluhovana->Activate();               // zruší odpočet exp(B), Wait skončí hned
    Activate(Time + Exponential(8 * 60));
  }
};

class Generator : public Event {             // „příchody transakcí“ ve smyčce s exp(A)
  void Behavior() {
    (new Transakce)->Activate();             // nová značka v místě „Fronta“
    Activate(Time + Exponential(A));
  }
};

int main() {
  Init(0, 60 * 100000);                      // modelový čas v minutách: 100 000 hodin
  (new Generator)->Activate();
  (new Porucha)->Activate(Exponential(8 * 60));
  Run();
  Linka.Output();
  Doba.Output();
  Print("Poruchy: %ld, zničené obsluhy: %ld\n", pocet_poruch, zniceno);
}
```

Petriho síť	SIMLIB
příchody transakcí ⇄ `exp(A)`	událost `Generator`, která se znovu plánuje za `Exponential(A)`
značka (transakce)	proces `Transakce`
Fronta + obsazení *Linky*	`Seize(Linka)` (fronta je vnitřní fronta zařízení)
místo *Linka* (1 značka)	`Facility Linka`
místo *obsluha*	ukazatel `obsluhovana` (`nullptr` = místo je prázdné)
`exp(B)` se sloučeným uvolněním *Linky*	`Wait(Exponential(B)); Release(Linka);`
vznik poruch ⇄ `exp(8 h)`	událost `Porucha`, která se znovu plánuje za `Exponential(8 * 60)`
místo *porucha*	čítač `porucha` (počet čekajících poruch)
zničení, když porucha přijde během obsluhy	`porucha++` a `obsluhovana->Activate()`: `Wait` skončí hned (odpočet `exp(B)` zanikne)
zničení, když porucha čeká	`if (porucha == 0)` hned po `Seize`: obsluha vůbec nezačne
výstupy zničení: *Linka* a *zničená* → *odchod bez obsluhy*	`Release(Linka)`, pak `porucha--` a `return`
přechod *odchod*	konec metody `Behavior()`

Porucha je tu událost, ne proces se `Seize(Linka, 1)`: nemá dobu trvání a přerušení v SIMLIBu (fronta přerušených) by obsluhu po uvolnění jen dokončilo.

Kontrola simulací. Petriho síť: *Linka* + *obsluha* = 1, všechny přechody se provádějí a každá porucha zničí právě jednu obsluhu (počet zničení = počet poruch); v místě *porucha* čekají nanejvýš dvě poruchy najednou (`make check`, tři běhy). SIMLIB model: v jednom běhu (100 000 hodin, A = 10 min, B = 6 min) vzniklo 12 239 poruch a stejný počet obsluh byl zničen, tedy 2,0 % transakcí. Teoreticky je to podíl intenzit poruch a příchodů: (1/480) / (1/10) = 2,1 % (`make simlib`).

Původ a zdroje

Původ: zkouška 2021/22, 2. termín, otázka 1 (body neuvedené).

Původní znění:

> Petri MM1 V intervaloch exp(8 hodín) sa objaví porucha ktorá buď: - zničí práve prebiehajúcu obsluhu - alebo počká na príchod následujúcej obsluhy a tú zničí
>
> + SIMLIB kód Modely MUSIA byť izomorfné.

Zdroje:

- `Fitcheats/Zkoušky z minulých let/2021-22 - 2. termín.txt`, otázka 1 (studentský přepis zadání)

Existující řešení:

- K této úloze žádné řešení není.